

Release Protocol

Invoked automatically via `CLAUDE.md`, which is loaded at the start of every session.

The purpose is narrow: **make sure nothing that describes the app is left saying something the app no longer does**. That is the failure this exists to prevent, and it has already happened once — the v1.2.1 training pack told staff photos lived in a folder the app had stopped using.

1. Classify the change first

Everything else follows from this. Decide before starting, not after.

Tier	Example	Version	Folder
Patch	Bug fix, wording fix, a test added	1.3.1 → 1.3.2	Stays <code>v1.3</code>
Minor	New field, new capability, visible behaviour change	1.3 → 1.4	New <code>v1.4</code> folder
Major	Schema change, new storage target, structural rework	1.x → 2.0	New <code>v2.0</code> folder

One folder per minor version, not per patch. Git holds every version and tags each release; copying the whole folder for a typo fix creates snapshots nobody reads and multiplies the documents that can drift. The folder is "the current app", named for its minor version.

Changed from the v1.2.1 rule, which required the folder to match `APP_VER` exactly. That rule predates the repository and now costs more than it returns.

2. What each tier requires

Every tier, without exception

- `tests.html` passes with zero failures
- `APP_VER`, `FORM_VER`, `VER_DATE` in `index.html` updated
- `CACHE_VERSION` in `sw.js` bumped — **the tests check this**
- `CHANGELOG.txt` entry, written as the change is made, not reconstructed after
- Committed with a message saying *why*, not just what
- Tagged in git — `v1.3.2`
- `Guides/` `PDFs` regenerated and renamed for the new version (§4a)

Minor — the above, plus

- `01_Setup` and `User Guide` — if anything changed about what staff tap or see
- `Training/` — cards, chart and module reviewed together, never singly
- `_Shared/Dryspace Context Brief.md` — version numbers and feature list (in the `_Shared` repository, not this one)
- `docs/DECISION-LOG.md` — any decision taken during the work
- New version folder created; previous moved to `Super seded`

Major — the above, plus

- `03_Handover and Version Control Protocol` — if the record's lifecycle changed
- `02_Iteration Guide` — if the rules for changing the code changed
- `_Shared/Field App Architecture Template.md` — if the architecture changed, since sibling apps are built from it (in the `_Shared` repository, not this one)
- Migration path written **and tested against a record from the previous version**
- Record schema number bumped, with an `ensureSchema()` step

Both `_Shared` documents live outside this repository, in `00_AI Tools in Development/_Shared/`, which is its own git repository. They describe how Dryspace builds *any* field app, not this one — see `docs/DECISION-LOG.md §4`. Updating them is still part of a release here; the commit just lands in a different repository.

3. The document matrix

Which documents describe what. Consult this when deciding what a change touches.

Document	Describes	Update when
<code>CHANGELOG.txt</code>	What changed, per version	Every change
<code>CLAUDE.md</code>	The non-negotiables	A rule is added or removed
<code>docs/DECISION-LOG.md</code>	Why things are as they are	Any decision is taken
<code>05_Release Protocol.md</code>	This process	The process changes
<code>_Shared/Field App Architecture Template.md</code>	The architecture, for sibling apps	Architecture changes
<code>01_Setup and User Guide</code>	What staff tap and see	Visible behaviour changes
<code>02_Iteration Guide</code>	How to change the code safely	The rules for changing code change
<code>03_Handover Protocol</code>	How the record moves between people	The record's lifecycle changes
<code>_Shared/Dryspace Context Brief.md</code>	Background for external tools	Version, features or terminology change
<code>Training/</code>	How staff are taught	Anything staff-facing changes

The training pack updates as a set. The cards, the chart and the module say the same things three ways. Editing one is how they come to contradict each other.

4. Output format

PDF only. The `.md`, `.svg` and `.html` files are the editable sources; PDFs are generated from them and **are committed alongside them**, because `tests.html` fetches each expected path over HTTP and fails when one is missing. Never edit a PDF — regenerate it, or the artefact and its source drift apart.

Most of them come from `tools\make_guides.py (§4a)`. Two do not, and are generated with the browser's Print → Save as PDF: `Training/Workflow_Chart.html` at A3 landscape, and `Training/Setup_and_Use_Deck.html` at A4 landscape. For both, **Background graphics ON**, or navy headers and coloured panels print as empty white boxes.

4a. Guides — the generated PDFs

Four documents are procedures a person follows away from a screen. Each has a PDF in `Guides/`, generated from the markdown and **never edited directly**:

Source (root)	Generated
01_Setup and User Guide.md	Guides/01_Setup and User Guide_v1.4.0.pdf
02_Iteration Guide.md	Guides/02_Iteration Guide_v1.4.0.pdf
03_Handover and Version Control Protocol.md	Guides/03_Handover and Version Control Protocol_v1.4.0.pdf
05_Release Protocol.md	Guides/05_Release Protocol_v1.4.0.pdf

The PDF carries the version; the markdown does not. That asymmetry is the whole point:

- The markdown is in git, which already knows every version. Renaming it each release would churn the history and break every link pointing at it.
- The PDF is a detached artefact someone may be holding weeks later. With the version in its name, staleness is visible to anyone — the app's home screen says `v1.4`, the PDF in your hand says `v1.3`, so it is out of date. No process and no memory required, just two numbers that do not match.

`tests.html` fetches each expected path and fails if it is missing, so a release cannot pass while the guides are stale.

Not given a PDF, deliberately: the two `_Shared` documents — the Dryspace Context Brief and the Field App Architecture Template — exist to be pasted into an AI, and a PDF makes copying harder; `docs/DECISION-LOG.md` grows continuously and would be stale within a week. The gap at 04 in the Guides folder is expected, not an omission.

Generating them

Open the markdown in a browser or editor and print to PDF. **Background graphics ON**, or coloured panels print as empty white boxes. Replace the previous version's PDF rather than keeping both — two versions on a shelf is how someone reads the wrong one.

5. Release sequence

- Tests pass — zero failures
- Version stamps all agree
- Documents for this tier updated (\$2), PDFs regenerated
- Commit and tag
- Push — **on GitHub Pages, pushing to `main` is the deployment**
- **Verify the served file actually changed** — fetch the live `index.html` and confirm `APP_VER` moved. Not that the push succeeded; that the app did
- Tell staff to fully close and reopen the app
- Move the previous version folder to `Superseded`

Step 6 is not optional

`v1.2.0` and `v1.2.1` were both completed and never deployed. Nobody noticed for two versions, because nothing ever checked that live matched intent.

Step 7 is only needed until `v1.3` is out

From `v1.3` the app detects its own updates and shows a banner. Before that it cannot — a device already running old code has no way to be told about new code.

6. What is automated, and what is not

Automated — `tests.html` fails the build if any of these are wrong:

- `CACHE_VERSION` does not carry `APP_VER`
- The field `id manifest` is missing or disagrees on version
- `CHANGELOG.txt` has no entry for the current version
- Any control lacks a `data-fid`, or any file input lacks a `data-mfid`
- A `v1.1.1` key has no migration path
- Upload verification, retry policy or state transitions behave incorrectly

Not automated — **still needs a person:**

- Whether the User Guide still describes what the app does
- Whether the training pack matches the workflow
- Whether a decision in the log has quietly been reversed
- Whether the deployed app is the one you meant to deploy

Worth building next

■ **Document freshness check** — compare each document's modified date against `index.html`, and flag any that is older than the code it describes. This is the largest remaining manual gap and is mechanically checkable.

■ **A pre-commit hook** that refuses a commit touching `index.html` without a corresponding `CHANGELOG.txt` change. Stronger than a checklist, because it cannot be forgotten.